

2025 年 CSP-J 模拟试题 1 入门级 C++语言试题

1~5: _____ 6~10: _____ 11~15: _____

程序阅读 1: _____

程序阅读 2: _____

程序阅读 3: _____

姓名: _____ 分数: _____

程序填空 1: _____

程序填空 2: _____

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 在人工智能领域，进行大量运算时，通常会选择具有强大并行处理能力的处理器()。能够高效地处理复杂的数据集和算法，从而加速计算过程。

- A. CPU
- B. GPU
- C. TPU
- D. APU

2. 十进制小数 15.25 对应的二进制数是()。

- A. 1110.01_2
- B. 1111.10_2
- C. 1111.01_2
- D. 111.01_2

3. 一个字长为 8 位的整数的补码是 11001011，则它的原码是()。

- A. 10011010
- B. 10110101
- C. 01101010
- D. 01111001

4. 字符串 "hello" 中，包含多少个不同的子串()

- A. 5
- B. 10
- C. 11
- D. 15

5. 由 0,1,1,4,5 可以组成()个没有重复的五位奇数。

- A. 120
- B. 60
- C. 30
- D. 27

6. 已知一棵完全二叉树中共有 1132 个结点，叶子结点的个数应为()。

- A. 109
- B. 566
- C. 567
- D. 1024

7. 在一个包含 n 个节点的双链表中，查找一个特定值的节点的时间复杂度通常是()。

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(\log n)$

8. 给定一个包含 n 个元素的数组，其中 n 为偶数，现在将数组分为前后两个部分，但这两部分内部元素是有序的。使用()算法能最快地将整个数组排序。

- A. 快速排序
- B. 归并排序
- C. 插入排序
- D. 桶排序

9. ()算法在平均情况下和最坏情况下都能达到 $O(n \log n)$ 的时间复杂度。

- A. 快速排序 B. 冒泡排序 C. 归并排序 D. 插入排序

10. 前缀表达式 $+ * 3 2 + 5 12$ 等价的中缀表达式是()

- A. $3 + 2 * 5 + 12$
 B. $3 + 2 * (5 + 12)$
 C. $3 * 2 + (5 + 12)$
 D. $3 * (2 + 5) + 12$

11. 序列 $a_1, a_2, \dots, a_n$ (n 为偶数)依次入栈, 以下哪个出栈序列在仅使用单个栈且要求每个元素都恰好出栈一次的情况下是不可能实现的? ()

- A. $a_2, a_1, a_4, a_3, \dots, a_n, a_{n-1}$ (交错出栈)
 B. $a_n, a_{n-1}, \dots, a_2, a_1$ (完全逆序)
 C. $a_1, a_2, a_3, a_4, \dots, a_{n-1}, a_n$ (与入栈顺序相同)
 D. $a_1, a_n, a_2, a_{n-1}, \dots, a_{n/2}, a_{n/2+1}$ (将元素分成两半并交替出栈)

12. 有 6 双不同颜色的袜子共 12 只袜子, 每双袜子左右脚各 1 只, 一次性从中取 4 只袜子。请问恰好能配成 1 双袜子的不同取法有几种? ()

- A. 240
 B. 270
 C. 540
 D. 1280

13. 下列程序输出结果为()

```
union Data {
    char c[5];
    int i;
    double d;
};
int main() {
    cout<< sizeof(union Data);
    return 0;
}
```

- A. 5
 B. 4
 C. 8
 D. 16

14. 已知有向图 G 的顶点集 $V = \{A, B, C, D, E\}$, 边集 $E = \{ \langle A, B \rangle, \langle A, C \rangle, \langle B, D \rangle, \langle C, D \rangle, \langle A, E \rangle, \langle E, D \rangle \}$, 则下列序列中, 哪个是 G 的拓扑排序? ()

- A. A, B, C, D, E
 B. A, B, E, C, D
 C. A, C, D, B, E
 D. A, C, B, D, E

15. 邓小平在()年明确提出“计算机的普及要从娃娃抓起”, 这一观点在中国计算机普及事业 和信息科技事业发展中起到了重要的推动作用, 同年 NOI 竞赛开始举办。

- A. 1981
 B. 1982
 C. 1983
 D. 1984

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填 T, 错误填 F; 除特殊说明外, 判断题 1.5 分, 选择题 3 分, 共计 40 分)

(1)

```
01 #include<bits/stdc++.h>
02 using namespace std;
```

```

03 struct point
04 {
05     int x, y, z;
06     bool operator <(const point& b)const
07     {
08         if (x != b.x) return x < b.x;
09         if (y != b.y) return y > b.y;
10         return z < b.z;
11     }
12 };
13 double dist1(point a, point b)
14 {
15     int xx = (a.x - b.x);
16     int yy = (a.y - b.y);
17     int zz = (a.z - b.z);
18     return sqrt(xx * xx + yy * yy + zz * zz);
19 }
20 int dist2(point a, point b)
21 {
22     int xx = abs(a.x - b.x);
23     int yy = abs(a.y - b.y);
24     int zz = abs(a.z - b.z);
25     return xx + yy + zz;
26 }
27 int n;
28 int main()
29 {
30     cin >> n;
31     vector<point> v(n);
32     for (int i = 0; i < n; i++)
33     {
34         cin >> v[i].x >> v[i].y >> v[i].z;
35     }
36     sort(v.begin(), v.end());
37     double sum1 = 0;
38     int sum2 = 0;
39     for (int i = 1; i < n; i++)
40     {
41         sum1 += dist1(v[i], v[i - 1]);
42         sum2 += dist2(v[i], v[i - 1]);
43     }
44     printf("%.3f %d", sum1, sum2);
45     return 0;
46 }

```

假设输入的 n 是不超过 100000 的正整数，输入的其余数是绝对值不超过 10000 的整数：

判断题

16. `dist1` 函数的作用是求两个点之间的欧几里得距离, `dist2` 函数的作用是求两个点之间的曼哈顿距离()
17. 若将第 15 行到第 17 行换成第 22 行到第 24 行的代码, 则输出的结果可能会错误。()
18. `sort` 排序的规则是先按照 x 升序, 再按照 y 升序, 最后按照 z 升序。()
19. 该段代码的时间复杂度为 $O(n \log n)$ 。()

单选题

20. (4 分) 以下说法正确的是()。
- A. `sum2` 和 `dist2` 也应该设置成 `double` 类型, 不然计算结果可能会出错
 - B. 如果改变排序规则, `sum1` 和 `sum2` 的值不会发生变化
 - C. 两个相同的点, 调用 `dist1` 算出来的结果不一定比 `dist2` 要小
 - D. 因为要保留三位小数输出, 所以只能使用 `printf` 进行输出, `cout` 没法保留小数
21. 当输入为 3 2 2 2 1 1 1 0 0 0 时, 输出的值为()。
- A. 3.464 6
 - B. 3.464 8
 - C. 3.121 8
 - D. 3.121 6

(2)

```

01 #include <bits/stdc++.h>
02 using namespace std;
03 int dp[105][105][105];
04 int tryFind(vector<string>& strs, int i, int m, int n)
05 {
06     if(i<0) return 0;
07     if(dp[i][m][n]!=-1) return dp[i][m][n];
08     int numsOf0 = 0;
09     int numsOf1 = 0;
10     string str = strs[i];
11     for(int j=0;j<str.size();j++)
12     {
13         if(str[j]=='0') numsOf0++;
14         else numsOf1++;
15     }
16     if(m>=numsOf0&& n>=numsOf1)
17     {
18         dp[i][m][n]=max(tryFind(strs,i-1,m,n), 1+tryFind(strs,i-1,m-numsOf0,n-numsOf1));
19     }else
20     {
21         dp[i][m][n]=tryFind(strs,i-1,m,n);
22     }
23     return dp[i][m][n];
24 }
25 int findMaxForm(vector<string>& strs, int m, int n)
26 {
27     if(strs.size()==0||(m==0&&n==0)) return 0;
28     memset(dp,-1,sizeof(dp));
29     return tryFind(strs,strs.size()-1,m,n);
30 }
31 int main()
32 {
33     int t,m,n;
34     cin>>t>>m>>n;
35     vector<string> strs(t);
36     for(int i=0;i<t;i++)
37         cin>>strs[i];
38     cout<<findMaxForm(strs,m,n);
39     return 0;
40 }

```

假设输入的 t, n, m 是不超过 100 的正整数, 输入的 $strs[i]$ 是长度不超过 600 且仅由 '0' 和 '1' 组成的字符串, 完成下面的判断题和单选题:

判断题

22. 若将第 7 行删去, 则程序在某些数据下的运行效率将会明显降低。()
23. 若将第 27 行删去, 则输出的结果可能会错误。()
24. 若 $t \leq n$ 、 $t \leq m$, 且 $strs[i]$ 长度为 1, 则输出的值为 t 。()

单选题

25. 该段代码的时间复杂度为(), 其中 $|s|$ 为字符串的长度。
 A. $O(n \times m \times |s|)$ B. $O(t \times n \times m \times |s|)$ C. $O(t! \times n \times m \times |s|)$ D. $O(t+n+m+|s|)$
26. 对于以下的输入数据, 输出结果为()。
 3 1 1
 10 0 1
 A. -1 B. 0 C. 1 D. 2
27. 对于以下的输入数据, 输出结果为()。
 5 5 3
 10 0001 111001 1 0
 A. 4 B. 0 C. 5 D. 3

5

(3)

```
01 #include <bits/stdc++.h>
02 using namespace std;
03 using ll = long long;
04 const int maxn=1e3+5;
05 int a[maxn],b[maxn],n;
06 ll e[maxn],ans;
07 void merge_sort(int left,int right)
08 {
09     if(left==right)return;
10     int mid=(left+right)/2;
11     merge_sort(left,mid),merge_sort(mid+1,right);
12     int i=left,j=mid+1,k=left;
13     e[mid+1]=0;
14     for(int i=mid;i>=left;i--) e[i]=e[i+1]+a[i];    //后缀和
15     while(i<=mid && j<=right)
16     {
17         if(a[i]<=a[j]) b[k++]=a[i++];
18         else
19         {
20             ans+=1ll*a[j]*(mid-i+1)+e[i];    //求逆序对代价
21             b[k++]=a[j++];
22         }
23     }
24     while(i<=mid) b[k++]=a[i++];
25     while(j<=right) b[k++]=a[j++];
26     for(int i=left;i<=right;i++) a[i]=b[i];
27 }
28 int main()
29 {
30     ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
31     cin>>n;
32     for(int i=1;i<=n;i++)
33     {
34         cin>>a[i];
35     }
36     merge_sort(1,n);
37     cout<<ans<<'\n';
38     return 0;
39 }
```

假设输入的 n 是不超过 1000 的正整数，输入的 $a[i]$ 是不超过 1000 的正整数，完成下面的判断题和单选题：

判断题

28. 上述代码实现了一种排序算法，可以将 a 数组按照从小到大的顺序排序。()
29. 若将第 20 行修改为 $ans+=a[j]*(mid-i+1)+e[i]$ ；则输出的结果可能会错误。()
30. 若依次输入 a 数组的值是不下降的，则输出的结果一定为 0。()

单选题

31. 该段代码的时间复杂度为()。
- A. $O(\sqrt{n})$ B. $O(n)$ C. $O(n\log n)$ D. $O(n^2)$
32. 对于以下的输入数据，输出结果为()。
- 3
8 3 8
A. 19 B. 11 C. 8 D. 1
33. 对于以下的输入数据，输出结果为()。
- 5
5 4 3 2 1
A. 15 B. 37 C. 55 D. 60

三、完善程序（单选题，每小题 3 分，共计 30 分）

(1) (小林吃苹果)问题: 小林喜欢吃苹果。这里有 N 堆苹果, 第 i 堆中有 a_i 个苹果。警卫已经离开了, 将在 H 小时后回来。小林每小时吃 K 个苹果。每个小时, 她将会选择一堆苹果, 从中吃掉 K 个。如果这堆苹果少于 K 个, 她将吃掉这堆的所有苹果, 然后这一小时内不会再吃更多的苹果。小林喜欢慢慢吃, 但仍然想在警卫回来前吃掉所有的苹果。求出小林每小时最少吃多少个苹果, 才能在警卫回来前吃完。试补全程序。

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 int n,h,l,r,k;
04 int apple[10005];
05 int check(int x)
06 {
07     int time=0;
08     for(int i=1;i<=n;i++)
09     {
10         int curTime=(①);
11         time+=curTime;
12     }
13     return (②);
14 }
15 int main()
16 {
17     cin>>n>>h;
18     for(int i=1;i<=n;i++)
19     {
20         cin>>apple[i];
21     }
22     l=1;
23     r=0;
24     for(int i=1;i<=n;i++)
25     {
26         r=(③);
27     }
28     k=r;
29     while(④){
30         int mid=(l+r)/2;
31         if(check(mid)){
32             l=mid+1;
33         }else{
34             k=mid;
35             r=(⑤);
36         }
37     }
38     cout<<k;
39     return 0;
40 }

```

34. ①处应填()。

- A. $apple[i] / x$
- B. $(apple[i] - 1) / x$
- C. $(apple[i] + x) / x$
- D. $(apple[i] + x - 1) / x$

35. ②处应填()

- A. $time > h$
- B. $time \geq h$
- C. $time < h$
- D. $time \leq h$

36. ③处应填()

- A. $apple[i]$
- B. i
- C. $\max(r, apple[i])$
- D. $\min(r, apple[i])$

37.④处应填()

- A. $l < r$
- B. $l \leq r$
- C. $l > r$
- D. $l \geq r$

38.⑤处应填()

- A. mid
- B. mid+1
- C. mid-1
- D. mid<<1

(2) (环形石子合并) 将 n 堆石子绕圆形操场排放, 现要将石子有序地合并成一堆。规定每次只能选择相邻的两堆石子合并成新的一堆, 并将新的一堆石子数记为该次合并的得分。试设计一个算法, 计算出将 n 堆石子合并成一堆的最小得分和最大得分。

试补全动态规划算法。

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N = 410, INF = 0x3f3f3f3f;
04 int a[N], sum[N], g[N][N], f[N][N];
05 int main()
06 {
07     int n;
08     cin >> n;
09     for(int i = 1; i <= n; i++)
10     {
11         cin >> a[i];
12         (①);
13     }
14     for(int i = 1; i <= (②) ; i++)
15     {
16         sum[i] = sum[i - 1] + a[i];
17     }
18     memset(f, 0x3f, sizeof f);
19     memset(g, -0x3f, sizeof g);
20     for(int len = 1; len <= n; len++)
21     {
22         for(int l = 1; (③); l++)
23         {
24             int r = l + len - 1;
25             if(l == r) g[l][r] = f[l][r] = 0;
26             else{
27                 for(int k = l; k < r; k++)
28                 {
29                     g[l][r] = (④);
30                     f[l][r] = min(f[l][r], f[l][k]+f[k+1][r]+sum[r]-sum[l-1]);
31                 }
32             }
33         }
34     }
35     int Max = -INF;
36     int Min = INF;
37     for(int i = 1; i <= n; i++)
38     {
39         Max = max(Max, g[i][i + n - 1]);
40         Min = (⑤);
41     }
42     cout << Min << endl;
43     cout << Max << endl;
44     return 0;
45 }
```


39.①处应填()。

- A. $a[i + 1] = a[i]$
- B. $a[i + 1] = a[i] * 2$
- C. $a[i + n] = a[i]$
- D. $a[i + n] = a[i] * 2$

40.②处应填()

- A. n
- B. $n + 1$
- C. $2 * n$
- D. N

41.③处应填()

- A. $l + len \leq n * 2$
- B. $l + len - 1 \leq n * 2$
- C. $l + len \leq n$
- D. $l + len - 1 \leq n$

42.④处应填()

- A. $\max(g[l][r], g[l][k] + g[k + 1][r] + \text{sum}[r] - \text{sum}[l - 1])$
- B. $\max(g[l][r], g[l][k] + g[k][r] + \text{sum}[r] - \text{sum}[l - 1])$
- C. $\max(g[l][r], g[l][k] + g[k + 1][r] + \text{sum}[r] - \text{sum}[l])$
- D. $\max(g[l][r], g[l][k] + g[k][r] + \text{sum}[r] - \text{sum}[l])$

43.⑤处应填()

- A. $\min(f[i][k], f[i][i + n - 1])$
- B. $\min(\text{Min}, f[i][i])$
- C. $\min(\text{Min}, f[i][i + 1])$
- D. $\min(\text{Min}, f[i][i + n - 1])$